

ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE FAKULTA STAVEBNÍ, OBOR GEODÉZIE A KARTOGRAFIE KATEDRA GEOMATIKY				
<i>název předmětu</i> GEOINFORMATIKA				
<i>číslo úlohy</i> 4	<i>název úlohy</i> Clusterizační algoritmy			
<i>školní rok</i> 2025/26	<i>skupina</i> 2	<i>zpracovali</i> Králič Adam, Matějková Barbora	<i>datum</i> 21. 12. 2025	<i>klasifikace</i>

Technická zpráva

1 Zadání úlohy

Úkolem bylo vytvořit vlastní implementaci algoritmu shlukování k-means. Vygenerovat alespoň 3 shluky ve 2D. Porovnat vlastní implementaci s využitím již vytvořené funkce k-means nad totožnými množinami bodů a případné rozdíly okomentovat.

Bonusovou úlohou je poté rozšířit implementaci do N-dimensionálního prostoru.

2 Popis úlohy

Shlukování je metoda identifikace podobných skupin v souboru dat. Hledají se taková řešení, při kterých se jednotlivé skupiny od sebe co nejvíce liší a zároveň, aby body v rámci jedné skupiny byly co nejpodobnější.

Obecně jde o nalezení struktury v datech, aniž by to bylo vázané na předem zadaný či známý výsledek.

Existují různé přístupy ke shlukové analýze.

- Sekvenční přístup – počet tříd (shluků) není znám, definuje se prahová hodnota a maximální počet shluků, zvýhodňuje kruhové shluky.
- Hierarchický přístup – v každém kroku se na základě předchozí kroku získá nové shlukování.
- Optimalizace nákladové funkce – např. optimalizace shlukových vzdáleností (použitá implementace), nejvyužívanější postup shlukové analýzy.

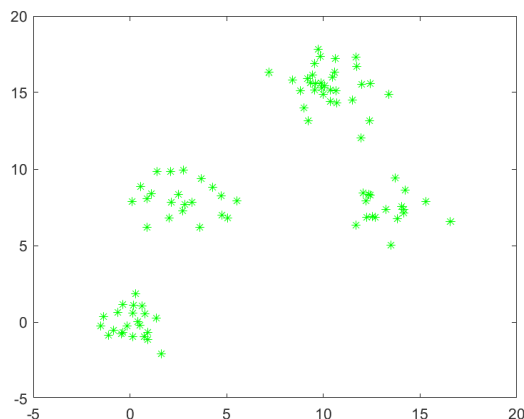
3 Vytvořené algoritmy

Zdrojový kód i použité skripty byly vytvořeny v programu Matlab R2023b.

3.1 Generování bodů

Body byly generovány náhodně v předdefinovaných shlucích, čehož bylo docíleno tak, že byly vytvořeny 4 náhodné množiny skládající se z různého počtu bodů, přičemž jejich centroidy byly systematicky posunuty (obr. 1).

```
A = randn(23,2);           % cluster A with 23 points
B = randn(18,2)*1.2+[13,7]; % cluster B with 18 points, + 13x + 7y
C = randn(32,2)*1.2+[10,15]; % cluster C with 32 points, + 10x + 15y
D = randn(21,2)*1.4+[3,8];  % cluster D with 21 points, + 3x + 8y
M = [A;B;C;D];             % union of clusters
```



Obrázek 1: Ukázka vygenerovaných počátečních bodů

3.2 Generování přibližných centroidů

Každý shluk je definován svým centroidem a pro správný chod algoritmu k-means je potřeba je nejprve přibližně definovat.

Existuje vícero možností, jak tyto prvotní centroidy nalézt.

1. Vlastní výběr – tento přístup není optimální, jelikož by algoritmus měl být nezávislý, každopádně tento přístup byl použit pro první pokusy implementace k-means shlukování.
2. Náhodný výběr – centroidy jsou vytvořeny zcela náhodně → může nastat nevhodná generace, při které budou všechny centroidy například příliš blízko sobě nebo budou obsaženy pouze v jednom sektoru.
3. Lineární inicializace – centroidy jsou umístěny lineárně po celé množině bodů, obecně poskytuje lepší výsledky, ale stále může vzniknout centroid v místech s nižší hustotou dat, použitá implementace.

4. K-means++ – umístění centroidů do bodu s největší vzdáleností od již umístěných centroidů.

V této práci byla nejdříve zvolena metoda vlastního výběru, jelikož je pro základní funkčnost algoritmu nejspolehlivější.

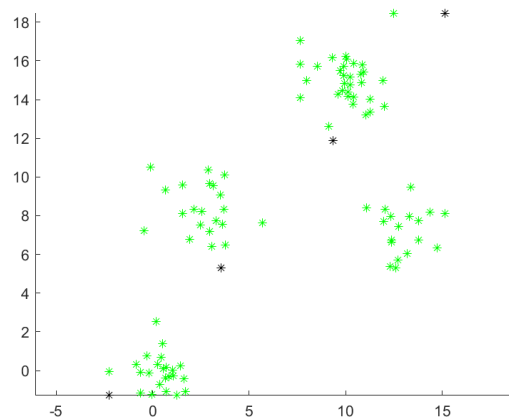
Program Matlab umožňuje pomocí metody `getpts` získat souřadnice bodů z aktuálně vykresleného grafu.

```
plot(M(:,1), M(:,2), "g*") % draws the point clusters
[xi,yi] = getpts;           % enables picking our own centroids
S = [xi, yi];               % stores the coordinates
```

Tento přístup je spolehlivý, ale není automatizovaný. Pro výslednou implementaci byla zvolena metoda lineární inicializace, při které jsou rovnoměrně generovány centroidy z levého spodního rohu do pravého horního (obr. 2).

```
S = []; % cluster coordinates
min_x = min(M(:,1)) % minimal x coordinate
min_y = min(M(:,2)) % minimal y coordinate
max_x = max(M(:,1)) % maximum x coordinate
max_y = max(M(:,2)) % maximum y coordinate
d_x = max_x-min_x % x difference
d_y = max_y-min_y % y difference

pocet_s = 4 % number of clusters
for i = 1:pocet_s % creates 4 centroid coordinates
    S(i,:) = [min_x + (d_x)*((i-1)/(pocet_s-1)), min_y + (d_y)*((i-1)/(pocet_s-1))]
end
```



Obrázek 2: ukázka lineární inicializace

3.3 Implementace k-means

Algoritmus využívá optimalizace shlukových vzdáleností, při které se v každé iteraci počítá ke každému bodu nejbližší centroid, a z těch samých množin nalezených bodů se následně vypočtou nové souřadnice centroidů.

Algoritmus probíhá, dokud se od sebe předchozí dvě iterace neliší (stejně souřadnice centroidů), nebo byl překročen zvolený maximální počet iterací.

```

Imax=20; % maximum number of iterations
I=0; % iteration number
S_prev = S - [1, 1]; % previous cluster coordinates
while I<Imax && ~isequal(S_prev,S) % until centroids are different or iteration limit exceed
    S_prev = S; % store previous centroids
    L = zeros(length(M(:,1)),1); % array of clousest centroid ids
    for i = 1:length(M) % for each point
        min_dist = inf; % sets infinite minimal dist
        closest_centroid_i = 0; % sets default closest centroid id
        for centroid_i = 1:length(S) % list through centroids
            d = dist(M(i,:), S(centroid_i,:)); % calculates dist. between point and cluster
            if d < min_dist % if point is closer to this centroid
                min_dist = d; % saves new minimal dist.
                closest_centroid_i = centroid_i; % updates closest centroid id
            end
        end
        L(i) = closest_centroid_i; % stores closest centroid id
    end

    %%% calculates new centroids %%%
    for centroid_i = 1:length(S) % for each cluster
        s_x = 0; % sum of each x
        s_y = 0; % sum of each y
        count = 0; % count of closest points
        for i = 1:length(M) % for each point in cloud
            if L(i) == centroid_i % if this centroid is closest to the point
                s_x = s_x + M(i,1); % add points x to sum
                s_y = s_y + M(i,2); % add points y to sum
                count = count + 1; % add to count of closest points
            end
        end

        if count > 0 % if there is atleast 1 closest point
            S(centroid_i, :) = [s_x/count, s_y/count]; % assigns new average x,y
        end
    end

    I = I + 1; % iteration + 1
end

```

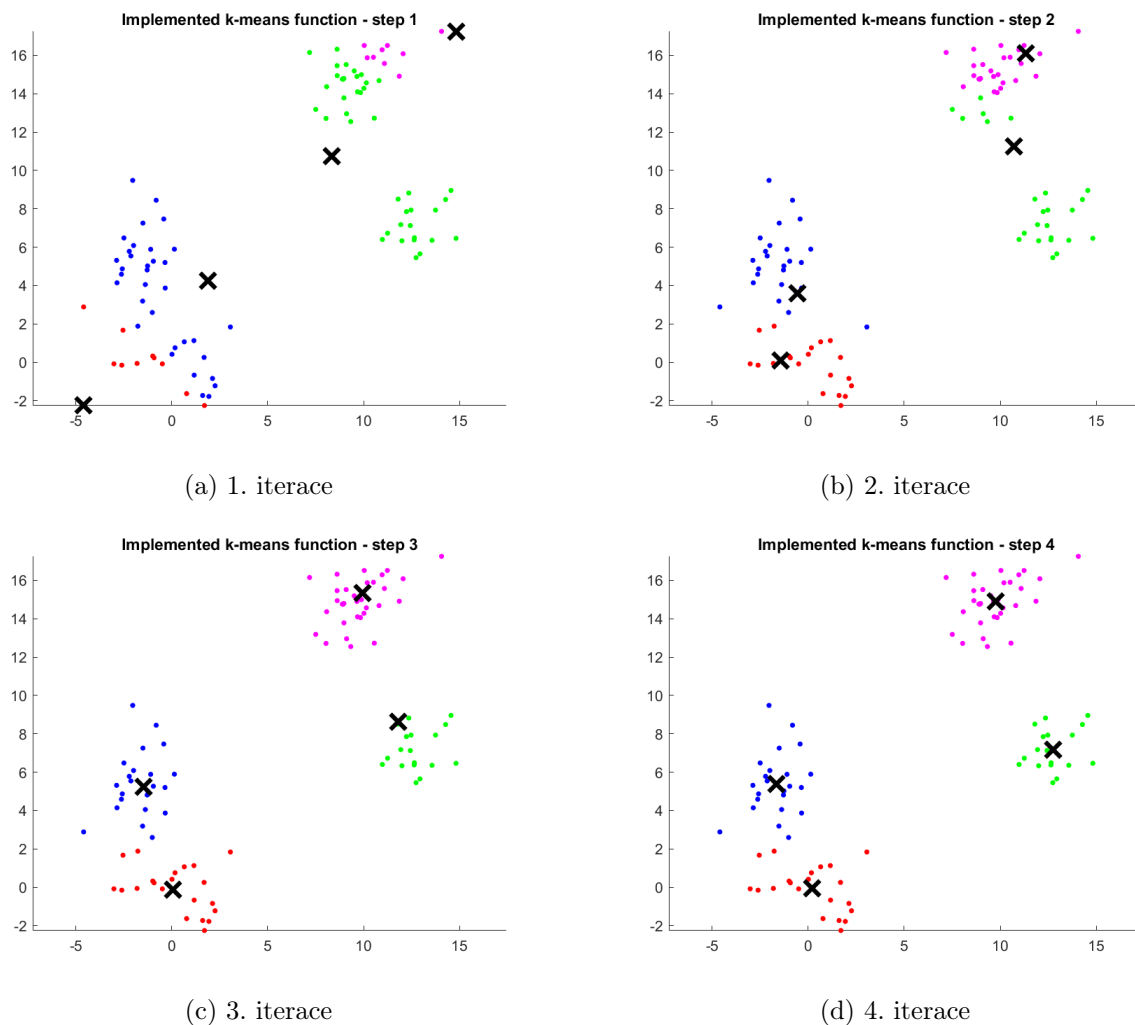
K této implementaci byla použita vytvořená funkce `dist(A, B)`, která počítá Eulerovskou vzdálenost mezi body A a B.

```

function d = dist(A, B)
d = sqrt((A(1)-B(1))^2 + (A(2)-B(2))^2);
end

```

Pro znázornění posunu mezi iteracemi a chodu algoritmu byl stav každé iterace vykreslen a uložen do .png souboru.



Obrázek 3: Implementace k-means – průběh iterací

4 Porovnání výsledků vlastní implementace a referenční funkce

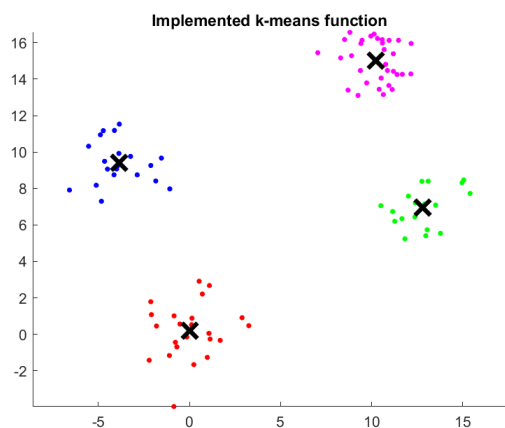
Celkově byly vytvořeny 3 případy shlukování se shluky o různých parametrech. Všechny tyto případy byly otestovány vlastní implementovanou metodou shlukování a vestavěnou funkcí `kmeans()` z programu Matlab.

Funkce `kmeans()` byla volána v její základní podobě. Na vstupu je množina použitých bodů M a žádaný počet vytvořených shluků. Funkce vrátí seznam bodů `idx` s příslušným id shluku, ve kterém se bod nachází a seznam souřadnic shluků C .

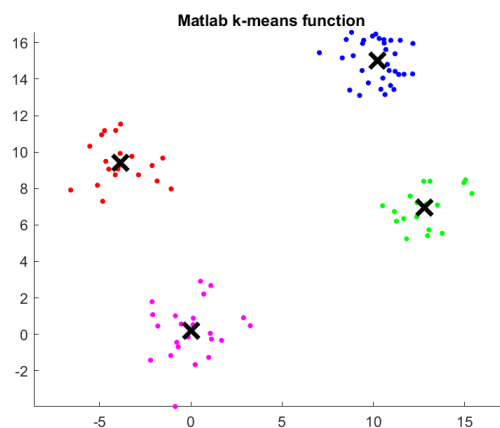
```
[idx,C] = kmeans(M,4);
```

4.1 1. případ – 4 menší, od sebe vzdálené shluky

Pro tento případ byly potřeba pouze 4 iterace a obě varianty jsou mezi sebou totožné.



(a) vlastní implementace

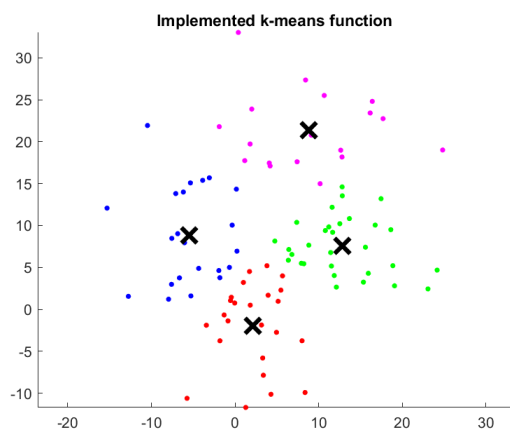


(b) funkce `kmeans()`

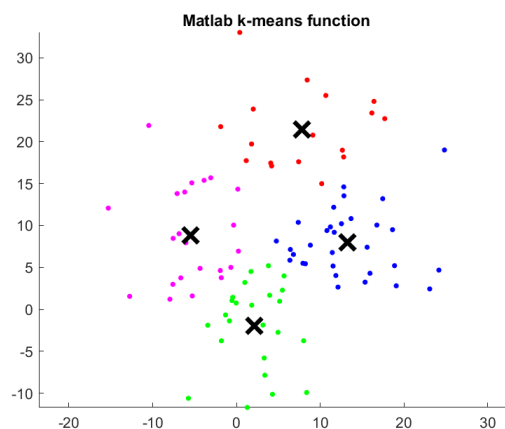
Obrázek 4: porovnání bodů 1. případu

4.2 2. případ – 4 rozlehlejší shluky, které se vzájemně překrývají

Pro tento případ bylo potřeba 11 iterací a u jednoho bodu dochází k rozdílu v zařazení (bod umístěný nejvíce vpravo).



(a) vlastní implementace

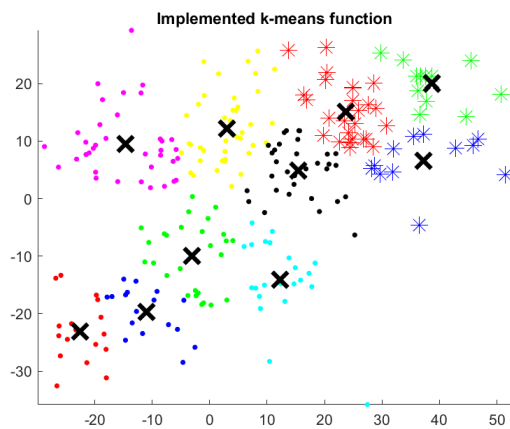


(b) funkce `kmeans()`

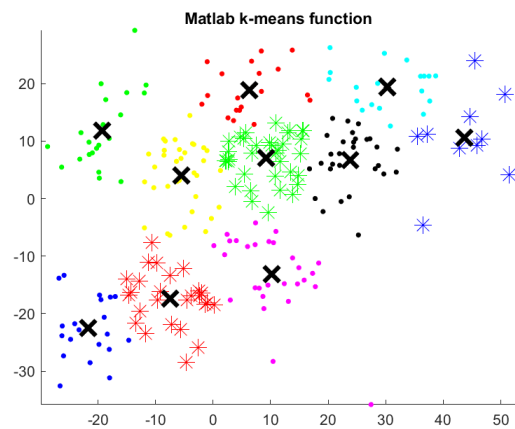
Obrázek 5: porovnání bodů 2. případu

4.3 3. případ – 10 rozlehlých shluků

Pro tento případ bylo potřeba 23 iterací a dochází zde k výrazným rozdílům, které již nejde jednoduše popsat, což je nejspíše způsobeno jiným prvotním rozložením centroidů a vyšším počtem bodů a tříd.



(a) vlastní implementace



(b) funkce `kmeans()`

Obrázek 6: porovnání bodů 3. případu

Závěr

Byla splněna pouze povinná část.

Byla implementována metoda k-means pomocí optimalizace shlukových vzdáleností v programu Matlab. Metoda je univerzální pro požadovaný počet shluků a rozložení bodů. Je omezená pouze pro práci ve 2D prostoru, nebyla tedy splněna bonusová úloha o rozšíření do obecně N-dimenzionálního prostoru.

Mohl by být vylepšen prvotní výpočet přibližných souřadnic shluků metodou k-means++, což by pro většinu složitějších případů nejspíše našlo jiné souřadnice shluků, jelikož metoda k-means vysoce závisí právě na prvotním určení těchto souřadnic.

Z našich výsledků lze pozorovat, že počet iterací se s početnějšími a hustějšími daty zvyšuje, pro jednoduché případy často stačí pouze pár iterací, kdežto u složitějších případů jich mohou být desítky.

Přílohy

- Příloha 1 – výpočetní skript `u5_kmeans.m`

Reference

- [1] Výklad ze cvičení – Ing. Markéta Potůčková, Ph.D.

Podepsáno dne 21. 12. 2025 v Praze

Králič Adam, Matějková Barbora